



Kantonsschule Im Lee

Ablauf und Ziele von heute

- ▶ Was bisher geschah? (previously im EF-Informatik)

Ablauf und Ziele von heute

- ▶ Was bisher geschah? (previously im EF-Informatik)
- ▶ Herleitung und Diskussion der berühmten Fibonacci-Folge

Ablauf und Ziele von heute

- ▶ Was bisher geschah? (previously im EF-Informatik)
- ▶ Herleitung und Diskussion der berühmten Fibonacci-Folge
- ▶ Umsetzung rekursiver Programme in Computern

Ablauf und Ziele von heute

- ▶ Was bisher geschah? (previously im EF-Informatik)
- ▶ Herleitung und Diskussion der berühmten Fibonacci-Folge
- ▶ Umsetzung rekursiver Programme in Computern
- ▶ diverse Anwendungen

rekursive Berechnung von 5!

$$5! = 5 \cdot \underbrace{4 \cdot 3 \cdot 2 \cdot 1}_{4!}$$

$$4! = 4 \cdot \underbrace{3 \cdot 2 \cdot 1}_{3!}$$

$$3! = 3 \cdot \underbrace{2 \cdot 1}_{2!}$$

$$2! = 2 \cdot \underbrace{1}_{1!}$$

$$1! = 1 \cdot \underbrace{0!}_1 = 1$$

rekursive Berechnung von 5!

Durch *Rückwertseinsetzen* erhalten wir nun den Wert für 5!:

$$1! = 1 \cdot 0! = 1$$

$$2! = 2 \cdot 1! = 2 \cdot 1 = 2$$

$$3! = 3 \cdot 2! = 3 \cdot 2 = 6$$

$$4! = 4 \cdot 3! = 4 \cdot 6 = 24$$

$$5! = 5 \cdot 4! = 5 \cdot 24 = 120.$$

rekursive Definition von $n!$

Es sei n eine natürliche Zahl. Dann definieren wir die Fakultät $n!$ von n durch

$$n! := \begin{cases} 1, & \text{falls } n = 0, \text{ (Rekursionsanfang)} \\ n(n-1)!, & \text{falls } n \geq 1. \text{ (Rekursionsschritt)} \end{cases}$$

rekursive Berechnung von $n!$

```
1 def factorial(n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return n * factorial(n-1)
```

Definition und Berechnung der Potenz a^n

Sei $a \in \mathbb{R}$ und n eine natürliche Zahl. Wir definieren den Potenzausdruck a^n rekursiv durch

$$a^n := \begin{cases} 1, & \text{falls } n = 0, \text{ (Rekursionsanfang)} \\ a \cdot a^{n-1}, & \text{falls } n \geq 1. \text{ (Rekursionsschritt)} \end{cases}$$

Definition und Berechnung der Potenz a^n

```
1 def potenz(a,n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return a * potenz(a,n-1)
```

Definition und Berechnung der Potenz a^n

```
1 def potenz(a,n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return a * potenz(a,n-1)
```

Summe aller Listeneinträge

In Python ist eine Liste L von reellen Zahlen gegeben. Schreiben Sie eine Python-Funktion `sum_rek(L)`, welche rekursiv die Summe der Zahlen in der Liste berechnet. Es kann angenommen werden, dass die Länge der Liste ≥ 1 ist.

```
1 def sum_rek(L):  
2     if len(L) == 1:  
3         return L[0]  
4     else:  
5         return sum_rek(L[:-1]) + L[-1]
```

Dabei ist `L[-1]` das hinterste Element der Liste und `L[:-1]` die gesamte Liste ohne das hinterste Element.

Palindrome

Ein Wort heisst *Palindrom*, falls das Wort vorwärts und rückwärts genau gleich gelesen wird.

Beispiele:

- ▶ neben
- ▶ Rentner
- ▶ Otto
- ▶ Lagerregal

Das *leere Wort*, also das Wort der Länge 0, ist ebenfalls ein Palindrom. Schreiben Sie ein rekursives Python-Programm, welches entscheidet, ob ein gegebenes Wort w ein Palindrom ist oder nicht.

Palindrome

```
1 def check_palindrom(w):  
2     w = w.upper()  
3     if len(w) <= 1:  
4         return True  
5     elif w[0] != w[-1]:  
6         return False  
7     return check_palindrom(w[1:-1])
```

Palindrome

Lösung von Schafer, Gondorf et al., 2023

```
1 def check_palindrom(w):  
2     return (True if len(w) <= 1 else (check_palindrom(w  
    [1:-1]) if w[0].lower() == w[-1].lower() else False))
```

Fibonacci-Folge

In der zweiten Fassung des Buches *Liber abbaci* (Buch der Rechenkunst) beschrieb der italienische Mathematiker *Leonardo da Pisa*, bekannt als *Fibonacci*, das Wachstum einer Kaninchenpopulation. Dies war im Jahre 1202.

Fibonacci-Folge

- ▶ Jedes geschlechtsreife Paar Kaninchen wirft pro Monat genau ein neues Paar Kaninchen (ein Weibchen und ein Männchen). Die Austragungszeit (Dauer der Schwangerschaft) dauert bei Kaninchen also immer genau einen Monat. Jeden Monat kommt also eine neue Generation von Kaninchen zur Welt.

Fibonacci-Folge

- ▶ Jedes geschlechtsreife Paar Kaninchen wirft pro Monat genau ein neues Paar Kaninchen (ein Weibchen und ein Männchen). Die Austragungszeit (Dauer der Schwangerschaft) dauert bei Kaninchen also immer genau einen Monat. Jeden Monat kommt also eine neue Generation von Kaninchen zur Welt.
- ▶ Ein neugeborenes Paar von Kaninchen wird erst am Ende seines ersten Lebensmonats geschlechtsreif und wirft entsprechend erst Ende seines zweiten Lebensmonats sein erstes Paar Kaninchen.

Fibonacci-Folge

- ▶ Jedes geschlechtsreife Paar Kaninchen wirft pro Monat genau ein neues Paar Kaninchen (ein Weibchen und ein Männchen). Die Austragungszeit (Dauer der Schwangerschaft) dauert bei Kaninchen also immer genau einen Monat. Jeden Monat kommt also eine neue Generation von Kaninchen zur Welt.
- ▶ Ein neugeborenes Paar von Kaninchen wird erst am Ende seines ersten Lebensmonats geschlechtsreif und wirft entsprechend erst Ende seines zweiten Lebensmonats sein erstes Paar Kaninchen.
- ▶ Kein Kaninchen stirbt, verlässt das betrachtete System oder wird, ausser durch Geburt, in das System hineingebracht.

Fibonacci-Folge

- ▶ Sei G_n die Anzahl der geschlechtsreifen Kaninchenpaare und g_n die Anzahl der nicht geschlechtsreifen Kaninchenpaare der Generation n für $n \in \mathbb{N}$.

Fibonacci-Folge

- ▶ Sei G_n die Anzahl der geschlechtsreifen Kaninchenpaare und g_n die Anzahl der nicht geschlechtsreifen Kaninchenpaare der Generation n für $n \in \mathbb{N}$.
- ▶ Beachten Sie, dass die gesamte Anzahl der Kaninchenpaare der Generation n damit der Summe $F_n := G_n + g_n$ entspricht.

Fibonacci-Folge

Sei $n \in \mathbb{N}$.

► Es gilt

$$G_{n+2} = G_{n+1} + g_{n+1},$$

da die geschlechtsreifen Kaninchen G_{n+1} der Generation $n + 1$ auch einen Monat später noch geschlechtsreif sein werden und die nicht geschlechtsreifen Kaninchen g_{n+1} der Generation $n + 1$ einen Monat später geschlechtsreif sein werden.

Fibonacci-Folge

Sei $n \in \mathbb{N}$.

- Es gilt

$$G_{n+2} = G_{n+1} + g_{n+1},$$

da die geschlechtsreifen Kaninchen G_{n+1} der Generation $n + 1$ auch einen Monat später noch geschlechtsreif sein werden und die nicht geschlechtsreifen Kaninchen g_{n+1} der Generation $n + 1$ einen Monat später geschlechtsreif sein werden.

- Völlig analog begründet man die Gleichung

$$G_{n+1} = G_n + g_n.$$

Fibonacci-Folge

Sei $n \in \mathbb{N}$.

- Es gilt

$$G_{n+2} = G_{n+1} + g_{n+1},$$

da die geschlechtsreifen Kaninchen G_{n+1} der Generation $n + 1$ auch einen Monat später noch geschlechtsreif sein werden und die nicht geschlechtsreifen Kaninchen g_{n+1} der Generation $n + 1$ einen Monat später geschlechtsreif sein werden.

- Völlig analog begründet man die Gleichung

$$G_{n+1} = G_n + g_n.$$

- Offensichtlich gilt zudem

$$g_{n+2} = G_{n+1}.$$

Fibonacci-Folge

Wir haben also drei Gleichungen für die Population:

$$G_{n+2} = G_{n+1} + g_{n+1} \quad (1)$$

$$G_{n+1} = G_n + g_n \quad (2)$$

$$g_{n+2} = G_{n+1} \quad (3)$$

Wir setzen nun Gleichung 2 in Gleichung 1 ein und erhalten:

$$\begin{aligned} G_{n+2} &= G_n + g_n + g_{n+1} \\ &\iff \\ \underbrace{G_{n+2} + g_{n+2}}_{F_{n+2}} &= \underbrace{G_{n+1} + g_{n+1}}_{F_{n+1}} + \underbrace{G_n + g_n}_{F_n}. \end{aligned}$$

Fibonacci-Folge

Für die Gesamtzahl der Population der Kaninchenpaare gilt also

$$F_{n+2} = F_{n+1} + F_n$$

für alle $n \in \mathbb{N}$. Für Generation $n = 0$ definieren wir $G_0 := 0$ und $g_0 := 0$. Zu Beginn, also in der Generation $n = 1$, wird ein erstes Paar von Kaninchen in das System eingeführt. Dieses erste Paar wird erst nach einem Monat geschlechtsreif. Es gilt also $G_1 = 0$ und $g_1 = 1$. Somit besteht die Generation $n = 2$ immer noch aus nur einem Paar Kaninchen: $G_2 = 1$ und $g_2 = 0$. Die Generation $n = 3$ aus zwei Paaren: $G_3 = 1$ und $g_3 = 1$. Logisch fortgeführt findet man die sogenannte *Fibonacci-Folge*.

$$F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, F_5 = 5, F_6 = 8, \\ F_7 = 13, F_8 = 21, F_9 = 34, F_{10} = 55, F_{11} = 89, \dots$$

Fibonacci-Folge

Die Fibonacci-Folge ist somit rekursiv definiert durch

$$F_n := \begin{cases} 0, & \text{falls } n = 0, \text{ (Rekursionsanfang)} \\ 1, & \text{falls } n = 1, \text{ (Rekursionsanfang)} \\ F_{n-1} + F_{n-2}, & \text{falls } n \geq 2. \text{ (Rekursionsschritt)} \end{cases}$$

Spirale



Aufgaben

- ▶ Aufgabe 3.11 (A3.7)
- ▶ Aufgabe 3.12 (A3.8) ← die Schnellen können hier die Zeit nutzen, um einen eindrucklichen Plot zu erstellen.

Fibonacci-Folge auf einem 8-Bit Computer :)

Technische Umsetzung rekursiver Programme

```
1 def binaryStrings(n, w = ''):
2     if n == 0:
3         print(w)
4         return
5     binaryStrings(n-1, w + '0')
6     binaryStrings(n-1, w + '1')
7     #return # optional
```

Binäre Strings ohne aufeinanderfolgende Einsen

Bearbeiten Sie Kapitel 4 (Binäre Strings ohne aufeinanderfolgende Einsen).